Q: Are all of these enough to get full marks in the exam?

A: NO. This is a practice sheet. Meaning, you can practice all you want using the questions from this sheet. However, doing well in exams depends upon your ability to understand a question, formulate an answer, and express it correctly. You see, these are humane skills which cannot be guaranteed by completing a practice sheet only. But yeah, Best of luck anyway.

Chapter 2 (Instructions: Language of the Computer)

Question - 1:

Construct the equivalent RISC-V code of the following C code. Once you have the RISC-V code, identify type of each instruction and encode them accordingly.

```
A[7] = A[2] + A[B[8]] + 10;
B[i] = A[3] - 8;
```

Base addresses of array A and B are in register X_{20} and X_{21} and i is in register X_{22}

Question - 2:

Construct the equivalent RISC-V code of the following C code.

```
for (i = 8; i > 0 ; i--) {
  if ( A[i] == i){
     A[2] = A [B[3]] ;
  }
}
```

Base addresses of array A and B are in register X_{20} and X_{21} . Also consider i is in register X_{22}.

Question - 3:

Construct the equivalent RISC-V code of the following C code.

```
if ( A[i] < i){
     A[2] = A [B[3]] ;
}
```

Base addresses of array A and B are in register X_{20} and X_{21} . Also consider i is in register X_{22}.

Question - 4:

Construct the equivalent RISC-V code of the following C code.

```
if ( A[3] != A[6]){
     if (A[3] == 0) {
          A[3] = A[3] + 2;
     }else{
          A[6] = A[6] / 16;
     }
}else{
     A[6] = A[6] * 8
}
```

Base addresses of array A and B are in register X_{20} and X_{21} .

Question - 5:

Construct the equivalent RISC-V code of the following C code.

```
Main () {
    int x = 0;
    int y = 9;
    int z = addition(x, y);
}
```

```
int addition (int a, int b) {
    int c = a + b;
    return c;
}
```

Variables x, y, z are stored in X_{20}, X_{21} and X_{22} registers. Argument x, y are passed using register X_{13}, X_{14}
Variable c from the addition function also uses register X_{21}

Question - 6:

Write RISC-V assembly code that checks if the number stored in register $\mathbf{X_{25}}$ is **even** or not. If **even** then store **1** in register $\mathbf{X_{26}}$ otherwise store **0**.

Question - 7:

ADD X_{25} , X_{25} , X_0 . Can you make this instruction faster? If yes, Write the updated instruction?

Question - 8:

Memory Location	Code	Line Number	Machine Code
	ADDI $\mathbf{X_5}$, X_0 , 5	1	
	ADDI $\mathbf{X_6}$, X_0 , 1	2	
	ADDI $\mathbf{X_{25}}$, X_0 , 0	3	
	Loop: BLT $\mathbf{X_5}$, $\mathbf{X_6}$, loopBreak	4	_____XXX_____XXXXXXX
	ADDI X_{25} , X_{25} , 1	5	
#7080	ADDI X_5 , X_5 , -1	6	
	BEQ $\mathbf{X_0}$, $\mathbf{X_0}$, Loop	7	_____XXX_____XXXXXXX
	loopBreak:	8	

a) What is the value of **PC** while executing line2? Answer: _____
b) Fill up the machine codes corresponding to line4 and line7 in the table above.

Question - 9:

Memory Location	Code	Line Number
	Loop:	
	SLLI $\mathbf{X_{10}}$, X_{22} , 3	1
	ADD $\mathbf{X_{10}}$, X_{10} , X_{25}	2
	LD $\mathbf{X_9}$, 0(X_{10})	3
	BNE $\mathbf{X_9}$, $\mathbf{X_{24}}$, Exit	4
#80016	ADDI X_{22} , X_{22} , 1	5
	BEQ $\mathbf{X_0}$, $\mathbf{X_0}$, Loop	6
	Exit:	

a. Fill up the memory locations.
b. Find the SB-type instructions from the above code and encode them accordingly. Given,
   I. opcode = $(103)_{10}$, funct3 = $(000)_2$ opcode for BEQ
   II. opcode = $(103)_{10}$, funct3 = $(001)_2$ opcode for BNE

Question - 10:

Write necessary RISC-V instructions to store the value (1111 1111 0000 1111 11)2 in X20 register.

Question - 11:

Show how the value 0xabcdef12 would be arranged in memory in RISC-V machine.

Question - 12:

For the RISC-V assembly instructions below, what is the corresponding C/high level statement?

slli x30, x5, 3 add x30, x10, x30 slli x31, x6, 3	Assume that the variables f, g, h, i, and j are assigned to registers x5, x6, x7,

add x31, x11, x31 ld x5, 0(x30) addi x12, x30, 8 ld x30, 0(x12) add x30, x30, x5 sd x30, 0(x31)	x28, and x29, respectively. Assume that the base address of the Arrays A and B are in registers x10 and x11, respectively.

Newly added problems:

[Solutions available]

Question - 13:

Construct the equivalent RISC-V code **without using the multiplication operation** of the following C code:

```
int n = (a * 19)
```

Assume: n in x21, a in x22 *[Hint: use slli]*

Question - 14:

Construct the equivalent RISC-V code of the following C code:

```
int n = 0;
for(int i = 5; i <= n; i--) {
  if(A[i] > 0 && A[i] < n){
            A[i] = A[i] - 1;
    }
}
```

Assume : A base address in x20, n in x21, i in x22

Question - 15:

3. Translate the following assembly to equivalent C code.

Assume : A base in x21, B base in x20, i in x22

```
slli x10, x22, 3
add  x10, x10, x20
```

```
ld   x5, 0(x10)
addi x5, x5, 1
sd   x5, 24(x21)
```

Question - 16:

```
bge x22, x21, Exit              \\ 0x015B1A67
        slli x5, x22, 3
        add x5, x20, x5
        ld x10, 0(x5)
        addi x10, x10, -1
        sd x10, 0(x5)
        addi x22, x22, 1
```

The machine code of the first instruction is given beside the instruction. Based on that, deduce where should the label "Exit" be put?

Question - 17:

Construct the equivalent RISC-V code of the following C code:

```
int max (int a, int b){
    if(a>b) return a;
    else return b;
}

int main(){
    int x = max(1000, 6000);
}
```

Assume x is in x21.

Question - 18:

Construct the equivalent **RISC-V assembly** for the following C statement:

A[5] = A[i] * 12 + B[i];

Assume, Base addresses of **A and B are in x20 and x21 respectively. The value of 'i' is in x22 and data size is 4 bytes.**

Question - 19:

Assume the value:

0xCAFEBABE12345678

is stored starting at memory address 0x2000. Show the byte order in memory for both little endian and big endian

Question - 20:

Given the following RISC-V instructions, write their **32-bit machine code** in binary. Put **Don't care(X) where necessary(As opcode, funct3 or funct7 value is not given).**

BEQ x6, x0, target1 (Assume target1 is 32 bytes ahead of the current PC.)

BNE x10, x11, target2 (Assume target2 is 64 bytes before the current PC.)

Question - 21:

Translate the following **C code** into **RISC-V assembly**. You must use the **LUI instruction**.

int x = 0x12345083;

int y = 25;

int z;

z = x + y;

Question - 22:

Translate the following **C code** into **RISC-V assembly**.

```
int multiply_add(int a, int b) {
   int c;
   c = a * 4;
   return c + b;
}

int main() {
   int x = 6;
   int y = 9;
```

```
    int z;

    z = multiply_add_stack(x, y);
}
```

Assume:

x → x20
y → x21
z → x22
Function arguments → x10, x11
Return value → x10
Stack pointer → x2
a → x13
b → x14
c → x15

NB: Store and restore x13-x15 in stack during procedure